

Scale-Up Architecture and Hardware Plan

Step 4 of the Opportunity Tracking Agent build plan

Hardware: shares the same Blackwell card as the ROI Agent

Short answer: no new hardware needed. The same RTX PRO 6000 Blackwell (96GB) sized for the ROI Agent comfortably covers this agent too, since the two agents will not realistically place concurrent peak load on it at the same time.

This agent's judgment layer (confidence calibration) is an even lighter workload than the ROI Agent's discount factor scoring: shorter context per call, and a comparable or lower query volume, since pipeline updates happen on a periodic cadence rather than continuously. If both agents ever need simultaneous local inference, vLLM's continuous batching handles multiple concurrent requests against the same loaded model on one card without needing a second GPU. The practical move is to stand up one shared local-inference service on the Blackwell card that both agents call, rather than duplicating the deployment.

As with the ROI Agent, the recommendation is to route judgment calls through a hosted frontier API (Claude, GPT, or Gemini class) as the primary path, and keep the Blackwell card as the data residency and cost control fallback. If self-hosting is required, use Western open-weight models only (Llama 3.x, Mistral/Mixtral, Gemma, or Command R+), never Qwen or DeepSeek, for the same reason raised on the call about Chinese-origin models and Western customer trust.

Model size (quantized)	VRAM needed	Example GPU	Example cloud instance	Rough cost
7-8B (Llama 3.1 8B, Mistral 7B)	~10-16 GB	NVIDIA L4 or A10G	AWS g6.xlarge / g5.xlarge; GCP g2-standard-4	~\$0.50-\$1.00/hr
13-34B (Mixtral 8x7B class)	~24-48 GB	NVIDIA L40S or A100 40GB	AWS g6e.xlarge; Lambda/RunPod A100 40GB	~\$1.25-\$2.50/hr
70B (Llama 3.3 70B class)	~40-48 GB (4-bit)	1x NVIDIA RTX PRO 6000 Blackwell (96GB), shared with the ROI Agent	On-prem workstation, no cloud rental needed	Sunk cost, electricity only
100B+ (Mistral Large 2, Llama 3.1 405B)	~65-200+ GB	2x A100/H100 80GB, or 1x RTX PRO 6000 Blackwell for the 123B class at 4-bit	AWS p4de/p5 fractional via Lambda Labs, RunPod, or CoreWeave	~\$4-\$12/hr for multi-GPU

From POC to a platform-integrated agent

The POC hard-codes its input as a JSON file standing in for the Funnel and ProjectTrack tabs, and runs the judgment layer as a single function call per project. Scaling it up follows the same pattern already established for the ROI Agent.

Expose the calc engine and judgment layer as two separate callable tools: `compute_pipeline_rollup(pipeline_rows)` returns Sales Revenue, Adjusted Revenue, and roll-ups by region and stage; `score_confidence(project_context)` returns the confidence calibration with rationale for a single project. The Workflow Manager calls both and routes the result to whichever human or downstream agent needs it.

A stage tracking service, running on a schedule, watches for projects that have not moved stage in an unusually long time relative to similar past projects, and for the moment a project's Design Status resolves to Mass Production or Lost, at which point it proposes moving that record accordingly (subject to the open question above about whether that move should be automatic or human confirmed).

This agent's most important integration point is the ROI Agent: once a project reaches Design Win, its Adjusted Revenue, broken out by year, is exactly the shape of input the ROI Agent's revenue forecast intake expects. Building that hand-off as a shared data contract from the start avoids the two agents drifting into inconsistent numbers for the same project.

The same audit log principle applies here as with the ROI Agent: every confidence adjustment and every human override belongs in an immutable log from day one, both because sales leadership will want to see why a number moved, and because that log is what eventually lets a submitter reliability model be built from real outcomes.

Data model notes specific to this agent

Unlike the ROI Agent, this agent's core object is not a single project evaluated once, but a pipeline of many projects moving through stages over time, several of which will resolve to nothing (Design Lost). The data model should treat stage history as a first class, append only log per project, not just a current-state field, so that time-in-stage and win-loss patterns can be analyzed later without having thrown away the history that would answer those questions.

The regional and quarterly roll-up logic (Sales Revenue and Adjusted Revenue by region, by quarter, by stage) should be implemented once as shared code, not duplicated between whatever replaces the Funnel tab and whatever replaces the FCST_Revenue tab, since those two views are ultimately aggregating the same underlying project records.